CS336 Assignment 1: Basics

Writeup

Contents

1. Problem (`unicode1`): Understanding Unicode (1 point) . . . 2
2. Problem (`unicode2`): Unicode Encodings (3 points) . . . 2
3. Problem (`train_bpe_tinystories`): BPE Training on TinyStories (2 points) . . . 3
4. Problem (`train_bpe_expts_owt`): BPE Training on OpenWebText (2 points) . . . 4
5. Problem (`tokenizer_experiments`): Experiments with tokenizers (4 points) . . . 4
6. Problem (`transformer_accounting`): Transformer LM resource accounting (5 points) . . . 5
7. Problem (`learning_rate_tuning`): Tuning the learning rate (1 point) . . . 8
8. Problem (`adamw_accounting`): Resource accounting for training with AdamW (2 points) . . . 9
9. Problem (`learning_rate`): Tune the learning rate (2 B200 hrs) (3 points) . . . 12
10. Problem (`batch_size_experiment`): Batch size variations (1 B200 hr) (1 point) . . . 18
11. Problem (`generate`): Generate text (1 point) . . . 23
12. Problem (`layer_norm_ablation`): Remove RMSNorm and train (0.5 B200 hrs) (1 point) . . . 23
13. Problem (`pre_norm_ablation`): Implement post-norm and train (0.5 B200 hrs) (1 point) . . . 25
14. Problem (`no_pos_emb`): Implement NoPE (0.5 B200 hrs) (1 point) . . . 26
15. Problem (`swiglu_ablation`): SwiGLU vs. SiLU (0.5 B200 hrs) (1 point) . . . 28
16. Problem (`main_experiment`): Experiment on OWT (2 B200 hrs) (2 points) . . . 29
17. Problem (`leaderboard`): Leaderboard (10 B200 hrs) (6 points) . . . 29

1. Problem (`unicode1`): Understanding Unicode (1 point)

Prompt

(a) What Unicode character does chr(0) return?

Deliverable: A one-sentence response.

(b) How does this character's string representation (__repr__()) differ from its printed representation?

Deliverable: A one-sentence response.

(c) What happens when this character occurs in text? It may be helpful to play around with the following in your Python interpreter and see if it matches your expectations:

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

Deliverable: A one-sentence response.

Answer

a. 是编号为 0 的一个字符 NUL，解释器会把这个转义到'\x00'展示出来。

b. repr 会把不可见或需要转义的字符用转义写法表示出来。print 的时候把实际包含的字符显示出来。而\x00 没有可见字形，所以看起来像没有显示。

c. "this is a test" + chr(0) + "string" 是转义后的形式，print 的话就是输出字符串里实际包含的字符，chr(0)这个字符仍然在字符串里，只是打印时不可见。

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

2. Problem (`unicode2`): Unicode Encodings (3 points)

Prompt

(a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.

Deliverable: A one-to-two sentence response.

(b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

Deliverable: An example input byte string for which `decode_utf8_bytes_to_str_wrong` produces incorrect output, with a one-sentence explanation of why the function is incorrect.

(c) Give a two-byte sequence that does not decode to any Unicode character(s).

Deliverable: An example, with a one-sentence explanation.

Answer

a. 因为对很多真实文本，UTF-16/UTF-32 会引入更多冗余字节，而 UTF-8 更省字节，训练 tokenizer 更省数据和计算。对 ASCII/英文尤其如此。

b. 这个函数把 UTF-8 按单个字节分别 decode，但很多 Unicode 字符需要多个字节共同表示。比如传入“牛”.encode(“utf-8”) 时（即 b’\xe7\x89\x9b’），会报错。因为\xe7 是 leading byte，后面必须跟满足 10xxxxxx 模式的 continuation bytes，无法单独 decode。

c. 比如说 b’\xe7\xe7′。第一个 0xe7 是 1110xxxx，表示它开头的是一个 3-byte UTF-8 字符；但整个串只有 2 个字节，而且第二个 0xe7 也不是合法 continuation byte（不是 10xxxxxx）。

3. Problem (`train_bpe_tinystories`): BPE Training on TinyStories (2 points)

Prompt

(a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. How much time and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Resource requirements: $\leqslant$ 30 minutes (no GPUs), $\leqslant$ 30 GB RAM

Hint: You should be able to get under 2 minutes for BPE training using multiprocessing during pre-tokenization and the following two facts:

(a) The `<|endoftext|>` token delimits documents in the data files.

(b) The `<|endoftext|>` token is handled as a special case before the BPE merges are applied.

Deliverable: A one-to-two sentence response.

(b) Profile your code. What part of the tokenizer training process takes the most time?

Deliverable: A one-to-two sentence response.

Answer

a. 大约用了 17 秒；用 /usr/bin/time -v 测得峰值内存为 194724 kB，约等于 195 MB RSS。词表中最长的 token 是 b’ accomplishment’，长度为 15 bytes，这很合理，因为 TinyStories 中这类带前导空格的高频英文词片段确实容易被合并成一个 token。

b. 从 profiling 结果看，训练时间主要花在 pre-tokenization 阶段，而不是 BPE merge 主循环。也就是说，最耗时的部分是对语料分块、正则预分词和统计 pretoken 频次，而 merge 阶段相对只占很小一部分时间。

4. Problem (`train_bpe_expts_owt`): BPE Training on OpenWebText (2 points)

Prompt

(a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements: $\leqslant$ 12 hours (no GPUs), $\leqslant$ 100 GB RAM

Deliverable: A one-to-two sentence response.

(b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

Deliverable: A one-to-two sentence response.

Answer

a. 在 OpenWebText 上训练得到的 32,000 词表中，最长 token 是：

```
b'\\xc3\\x83\\xc3\\x82\\xc3\\x83\\xc3\\x82\\xc3\\x83\\xc3\\x82
  \\xc3\\x83\\xc3\\x82\\xc3\\x83\\xc3\\x82\\xc3\\x83\\xc3\\x82
  \\xc3\\x83\\xc3\\x82\\xc3\\x83\\xc3\\x82\\xc3\\x83\\xc3\\x82
  \\xc3\\x83\\xc3\\x82\\xc3\\x83\\xc3\\x82\\xc3\\x83\\xc3\\x82
  \\xc3\\x83\\xc3\\x82\\xc3\\x83\\xc3\\x82'
```

解码后对应一串 ÃÂÃÂ…。这基本可以解释为网页语料中的编码污染或乱码，放在 OpenWebText 这种抓取语料里是合理的。

b. 和 TinyStories 相比，OpenWebText 训练出来的 tokenizer 明显更“脏”也更多样，除了自然语言里的常见词片段，还学到了乱码、异常标点和网页文本噪声；而 TinyStories 的词表更干净，更偏向常见叙事文本中的高频词和带前导空格的英文词片段。

5. Problem (`tokenizer_experiments`): Experiments with tokenizers (4 points)

Prompt

(a) Sample 10 documents from TinyStories and OpenWebText. Using your previously trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary sizes, respectively), encode these sampled documents into integer ids. What is each tokenizer's compression ratio, measured in bytes per token?

Deliverable: A one-to-two sentence response.

(b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

Deliverable: A one-to-two sentence response.

(c) Estimate the throughput of your tokenizer, for example in bytes per second. About how long would it take to tokenize the Pile dataset (825 GB of text)?

Deliverable: A one-to-two sentence response.

(d) Using your TinyStories and OpenWebText tokenizers, encode the corresponding training and development datasets into sequences of integer token ids. We recommend storing the token ids as a NumPy array with dtype `uint16`. Why is `uint16` an appropriate choice?

Deliverable: A one-to-two sentence response.

Answer

a. 我在 TinyStories 和 OpenWebText 中各随机采样了 10 个文档。TinyStories tokenizer 在 TinyStories 样本上的压缩率约为 4.13 bytes/token，OpenWebText tokenizer 在 OpenWebText 样本上的压缩率约为 4.46 bytes/token；bytes/token 越高，表示每个 token 承载的原始字节越多，压缩效果越强。

b. 当我用 TinyStories tokenizer 去编码 OpenWebText 样本时，压缩率从 OpenWebText tokenizer 的约 4.46 bytes/token 降到了约 3.27 bytes/token。这说明 OpenWebText 文本会被切得更碎、需要更多 token，原因大概率是 TinyStories tokenizer 的词表更小，而且训练语料和 OpenWebText 的领域分布不匹配。

c. 我测得 tokenizer 的吞吐大约是 4 MB/s 左右。按这个速度粗略估算，tokenize 大约 825GB 的 The Pile 需要约 59-60 小时，实际时间还会受到硬件、I/O 和实现细节影响。

d. uint16 的取值范围为 0 至 65,535，足以覆盖本项目中 10,000 (10K) 和 32,000 (32K) 的词表大小；同时相比于 uint32 或默认的 int64，它能在保证不溢出的前提下将内存和存储开销减半，显著提高大规模数据集的处理效率。

6. Problem (`transformer_accounting`): Transformer LM resource accounting (5 points)

Prompt

(a) Consider a GPT-2 XL-sized model using our assignment architecture, which has the following configuration:

- `vocab_size: 50,257`
- `context_length: 1,024`
- `num_layers: 48`
- `d_model: 1,600`
- `num_heads: 25`
- `d_ff: 4,288` (the nearest multiple of 64 to `8 / 3 × 1,600`)

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

Deliverable: A one-to-two sentence response.

(b) Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that your input sequence has `context_length` tokens.

Deliverable: A list of matrix multiplies, with descriptions, and the total number of FLOPs required.

(c) Based on your analysis above, which parts of the model require the most FLOPs?

Deliverable: A one-to-two sentence response.

(d) Repeat your analysis with GPT-2 small (`12` layers, `768 d_model`, `12` heads), GPT-2 medium (`24` layers, `1024 d_model`, `16` heads), and GPT-2 large (`36` layers, `1280 d_model`, `20` heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

Deliverable: For each model, provide a breakdown of model components and their associated FLOPs as a proportion of the total FLOPs required for a forward pass. In addition, provide a one-to-two sentence description of how varying the model size changes the proportional FLOPs of each component.

(e) Take GPT-2 XL and increase the context length to `16,384`. How does the total FLOPs for one forward pass change? How does the relative contribution of FLOPs of the model components change?

Deliverable: A one-to-two sentence response.

Answer

a.

```
trainable parameters = 1640452800
```

single-precision floating point 是 32 bit 1640452800 * 32bit = 52494489600bit = 6561811200 B = 6.56GB

b. 只统计 matrix multiplies

```
total FLOPs: 3,516,769,894,400
transformer_blocks(95.32%): 3,352,087,756,800
|- Multihead_Self_Attention(37.78%): 1,328,755,507,200
  |- q_proj(7.16%): 251,658,240,000
  |- k_proj(7.16%): 251,658,240,000
  |- v_proj(7.16%): 251,658,240,000
  |- output_proj(7.16%): 251,658,240,000
  |- QK^T(4.58%): 161,061,273,600
  |- softmax(QK^T)V(4.58%): 161,061,273,600
|- FFN(57.53%): 2,023,332,249,600
  |- w1(19.18%): 674,444,083,200
  |- w2(19.18%): 674,444,083,200
  |- w3(19.18%): 674,444,083,200
lm_head(4.68%): 164,682,137,600
```

c. 在 GPT-2 XL、L=1024 下，FFN/SwiGLU 是最大的 FLOPs 来源，其次是 multi-head attention，lm_head 占比较小。

d.

```
GPT-2-small(trainable_parameters: 162,148,608):
total FLOPs: 291,648,307,200
transformer_blocks(72.90%): 212,600,881,152
|- Multihead_Self_Attention(33.13%): 96,636,764,160
  |- q_proj(4.97%): 14,495,514,624
  |- k_proj(4.97%): 14,495,514,624
  |- v_proj(4.97%): 14,495,514,624
  |- output_proj(4.97%): 14,495,514,624
  |- QK^T(6.63%): 19,327,352,832
  |- softmax(QK^T)V(6.63%): 19,327,352,832
|- FFN(39.76%): 115,964,116,992
  |- w1(13.25%): 38,654,705,664
  |- w2(13.25%): 38,654,705,664
  |- w3(13.25%): 38,654,705,664
lm_head(27.10%): 79,047,426,048

GPT-2-medium(trainable_parameters: 406,539,264):
total FLOPs: 830,172,299,264
transformer_blocks(87.30%): 724,775,731,200
|- Multihead_Self_Attention(37.25%): 309,237,645,312
  |- q_proj(6.21%): 51,539,607,552
  |- k_proj(6.21%): 51,539,607,552
  |- v_proj(6.21%): 51,539,607,552
  |- output_proj(6.21%): 51,539,607,552
  |- QK^T(6.21%): 51,539,607,552
  |- softmax(QK^T)V(6.21%): 51,539,607,552
|- FFN(50.05%): 415,538,085,888
  |- w1(16.68%): 138,512,695,296
  |- w2(16.68%): 138,512,695,296
  |- w3(16.68%): 138,512,695,296
lm_head(12.70%): 105,396,568,064

GPT-2-large(trainable_parameters: 833,591,040):
total FLOPs: 1,768,530,903,040
transformer_blocks(92.55%): 1,636,785,192,960
|- Multihead_Self_Attention(38.25%): 676,457,349,120
  |- q_proj(6.83%): 120,795,955,200
  |- k_proj(6.83%): 120,795,955,200
  |- v_proj(6.83%): 120,795,955,200
  |- output_proj(6.83%): 120,795,955,200
  |- QK^T(5.46%): 96,636,764,160
  |- softmax(QK^T)V(5.46%): 96,636,764,160
|- FFN(54.30%): 960,327,843,840
  |- w1(18.10%): 320,109,281,280
  |- w2(18.10%): 320,109,281,280
  |- w3(18.10%): 320,109,281,280
lm_head(7.45%): 131,745,710,080
```

随着模型规模在固定 context_length 下增大，Transformer blocks 的 FLOPs 占比上升，lm_head 占比下降。块内主要增长的是 FFN 和 attention 中的 projection matmuls，而真正的序列二次项 QK^T 与 softmax(QK^T)V 的相对占比反而下降。

e.

```
context_length: 16,384
total FLOPs: 133,577,729,638,400
transformer_blocks(98.03%): 130,942,815,436,800
|- Multihead_Self_Attention(73.79%): 98,569,499,443,200
  |- q_proj(3.01%): 4,026,531,840,000
  |- k_proj(3.01%): 4,026,531,840,000
  |- v_proj(3.01%): 4,026,531,840,000
  |- output_proj(3.01%): 4,026,531,840,000
  |- QK^T(30.87%): 41,231,686,041,600
  |- softmax(QK^T)V(30.87%): 41,231,686,041,600
|- FFN(24.24%): 32,373,315,993,600
  |- w1(8.08%): 10,791,105,331,200
  |- w2(8.08%): 10,791,105,331,200
  |- w3(8.08%): 10,791,105,331,200
lm_head(1.97%): 2,634,914,201,600
```

相比较之前的:

```
context_length: 1,024
total FLOPs: 3,516,769,894,400
transformer_blocks(95.32%): 3,352,087,756,800
|- Multihead_Self_Attention(37.78%): 1,328,755,507,200
  |- q_proj(7.16%): 251,658,240,000
  |- k_proj(7.16%): 251,658,240,000
  |- v_proj(7.16%): 251,658,240,000
  |- output_proj(7.16%): 251,658,240,000
  |- QK^T(4.58%): 161,061,273,600
  |- softmax(QK^T)V(4.58%): 161,061,273,600
|- FFN(57.53%): 2,023,332,249,600
  |- w1(19.18%): 674,444,083,200
  |- w2(19.18%): 674,444,083,200
  |- w3(19.18%): 674,444,083,200
lm_head(4.68%): 164,682,137,600
```

自注意力层明显占比上升，尤其是计算 QK^T 和 softmax(QK^T)V 的部分。

7. Problem (`learning_rate_tuning`): Tuning the learning rate (1 point)

Prompt

As we will see, one of the hyperparameters that affects training the most is the learning rate. Let's see that in practice in our toy example.

Run the SGD example above with three other values for the learning rate, `1e1`, `1e2`, and `1e3`, for just 10 training iterations. What happens with the loss for each of these learning rates? Does it decay faster, slower, or does it diverge, that is, increase over the course of training?

Deliverable: A one-to-two sentence response with the behaviors you observed.

Answer

- `lr = 1e1`: loss 稳定下降。
- `lr = 1e2`: loss 下降得更快，并迅速接近 0。
- `lr = 1e3`: loss 在 10 次迭代中快速增大，表现出发散趋势。

8. Problem (`adamw_accounting`): Resource accounting for training with AdamW (2 points)

Prompt

Let us compute how much memory and compute running AdamW requires. Assume we are using `float32` for every tensor.

(a) How much peak memory does running AdamW require? Decompose your answer based on the memory usage of the parameters, activations, gradients, and optimizer state. Express your answer in terms of `batch_size` and the model hyperparameters (`vocab_size`, `context_length`, `num_layers`, `d_model`, `num_heads`). Assume `d_ff = 8 / 3 × d_model`.

For simplicity, when calculating memory usage of activations, consider only the following components:

- Transformer block: RMSNorm(s)
- Transformer block: multi-head self-attention sublayer (`QKV` projections, `QK^T` matrix multiply, softmax, weighted sum of values, output projection)
- Transformer block: position-wise feed-forward (SwiGLU): `W1`, `W2`, SiLU on the gate branch, element-wise product, `W3`
- Final RMSNorm
- Output embedding
- Cross-entropy on logits

Deliverable: An algebraic expression for each of parameters, activations, gradients, and optimizer state, as well as the total.

(b) Instantiate your answer for a GPT-2 XL-shaped model to get an expression that only depends on `batch_size`. What is the maximum batch size you can use and still fit within `80 GB` memory?

Deliverable: An expression that looks like `a · batch_size + b` for numerical values `a`, `b`, and a number representing the maximum batch size.

(c) How many FLOPs does running one step of AdamW take?

Deliverable: An algebraic expression, with a brief justification.

(d) Model FLOPs utilization (MFU) is defined as the ratio of observed throughput, tokens per second, relative to the hardware's theoretical peak FLOP throughput [A. Chowdhery et al., 2022]. An NVIDIA H100 GPU has a theoretical peak of `495 teraFLOP/s` for "float32", actually TensorFloat-32, which in reality is "bfloat19", operations. Assuming you are able to get `50% MFU`, how long would it take to train a GPT-2 XL for `400K` steps and a batch size of `1024` on a single H100? Following J. Kaplan et al. [25] and J. Hoffmann et al. [26], assume that the backward pass has twice the FLOPs of the forward pass.

Deliverable: The number of hours training would take, with a brief justification.

Answer

a.

- input embedding
  - parameters
    - `vocab_size * d_model`
  - activations(not consider in this problem)
  - gradients
    - `vocab_size * d_model`

- optimizer state
  - `2 * (vocab_size * d_model)`
- Transformer block
  - 2 * RMSNorm
    - parameters
      - `2 * (d_model)`
    - activations
      - rms
        - `2 * (batch_size * context_length * 1)`
      - output
        - `2 * (batch_size * context_length * d_model)`
    - gradients
      - `2 * (d_model)`
    - optimizer state
      - `2 * (2 * d_model)`
  - Multi-head self-attention sublayer
    - `QKV` projections
      - parameters
        - `3 * (d_model * d_model)`
      - activations
        - `3 * (batch_size * context_length * d_model)`
      - gradients
        - `3 * (d_model * d_model)`
      - optimizer state
        - `6 * (d_model * d_model)`
    - `QK^T` matrix multiply (Q: [batch_size num_heads context_length hd_k], K: [batch_size num_heads context_length hd_k])
      - activations
        - `batch_size * num_heads * context_length * context_length`
    - softmax
      - activations
        - `batch_size * num_heads * context_length * context_length`
    - weighted sum of values([batch_size num_heads context_length context_length], [batch_size num_heads context_length hd_v])
      - activations
        - `batch_size * context_length * d_model`
    - output projection
      - parameters
        - `d_model * d_model`
      - activations
        - `batch_size * context_length * d_model`
      - gradients
        - `d_model * d_model`
      - optimizer state
        - `2 * d_model * d_model`
  - Position-wise feed-forward (SwiGLU)
    - gate projection
      - parameters
        - `d_model * d_ff`
      - activations

        - batch_size * context_length * d_ff
    - gradients
        - d_model * d_ff
    - optimizer state
        - 2 * d_model * d_ff
  - value projection
    - parameters
        - d_model * d_ff
    - activations
        - batch_size * context_length * d_ff
    - gradients
        - d_model * d_ff
    - optimizer state
        - 2 * d_model * d_ff
  - SiLU on the gate branch
    - activations
        - batch_size * context_length * d_ff
  - element-wise product
    - activations
        - batch_size * context_length * d_ff
  - down projection
    - parameters
        - d_model * d_ff
    - activations
        - batch_size * context_length * d_model
    - gradients
        - d_model * d_ff
    - optimizer state
        - 2 * d_model * d_ff
- final RMSNorm
  - parameters
    - d_model
  - activations
    - rms
      - batch_size * context_length * 1
    - output
      - batch_size * context_length * d_model
  - gradients
    - d_model
  - optimizer state
    - 2 * d_model
- Output embedding
  - parameters
    - d_model * vocab_size
  - activations
    - batch_size * context_length * vocab_size
  - gradients
    - d_model * vocab_size
  - optimizer state
    - 2 * d_model * vocab_size

- Cross-entropy on logits
  - activations
    - max-logits
      - `batch_size * context_length`
    - exp_shifted
      - `batch_size * context_length * vocab_size`
    - exp_shifted_sum
      - `batch_size * context_length`
    - correct_logits
      - `batch_size * context_length`

```
parameters memory:
  4*D*(2*L*(6*D + 1) + 2*V + 1)

activations memory:
  4*B*T*(3*D + 2*L*(28*D + 3*H*T + 3) + 6*V + 12)/3

gradients memory:
  4*D*(2*L*(6*D + 1) + 2*V + 1)

optimizer_state memory:
  8*D*(2*L*(6*D + 1) + 2*V + 1)

total memory:
  4*B*T*(3*D + 2*L*(28*D + 3*H*T + 3) + 6*V + 12)/3
  + 16*D*(2*L*(6*D + 1) + 2*V + 1)
```

b.

GPT-2 XL-shaped model memory: `16357023744*B + 26168601600 bytes`

Maximum batch size within 80GB memory: `3`

c. AdamW optimizer update FLOPs: 14 * num_parameters

d. about 4850 hours

9. Problem (`learning_rate`): Tune the learning rate (2 B200 hrs) (3 points)

Prompt

The learning rate is one of the most important hyperparameters to tune. Taking the base model you've trained, answer the following questions.

(a) Perform a hyperparameter sweep over the learning rates and report the final losses, or note divergence if the optimizer diverges.

Deliverable: Learning curves associated with multiple learning rates. Explain your hyperparameter search strategy.

Deliverable: A model with validation loss, per token, on TinyStories of at most `1.45`.

Low-Resource Tip: Train for a few steps on CPU or Apple Silicon

If you are running on `cpu` or `mps`, you should instead reduce the total tokens processed count to `40,000,000`, which will be sufficient to produce reasonably fluent text. You may also increase the target validation loss from `1.45` to `2.00`.

Running our solution code with a tuned learning rate on an M4 Max chip and `36 GB` of RAM, we use `batch size × total step count × context length = 32 × 5000 × 256 = 40,960,000` tokens, which takes `1 hour and 22 minutes` on `cpu` and `36 minutes` on `mps`. At step `5000`, we achieve a validation loss of `1.80`.

Some Additional Tips

- When using `N` training steps, we suggest adjusting the cosine learning rate decay schedule to terminate its decay, that is, reach the minimum learning rate, at precisely step `N`.
- When using `mps`, do not use TF32 kernels. In particular, do not set:

```
torch.set_float32_matmul_precision("high")
```

as you might with CUDA devices. We tried enabling TF32 kernels with `mps`, using `torch` version `2.9.0`, and found the backend sometimes uses silently broken kernels that cause unstable training.

- You can speed up training by JIT-compiling your model with `torch.compile`.

Specifically:

```
# On cpu
model = torch.compile(model)

# On mps
model = torch.compile(model, backend="aot_eager")
```

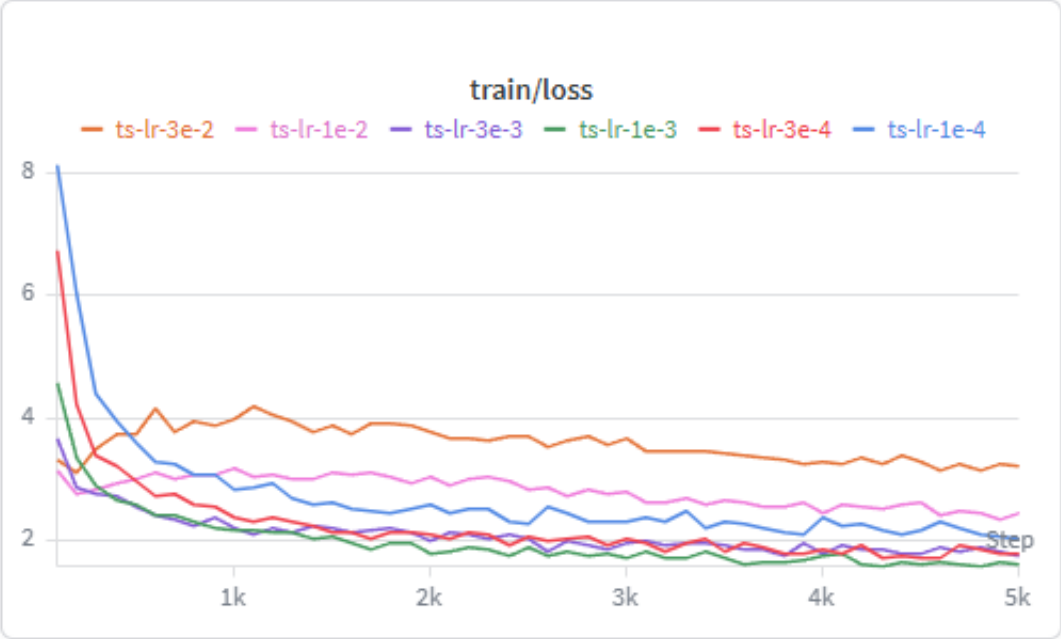
Compilation with Inductor is not supported on `mps` as of `torch` version `2.9.0`.

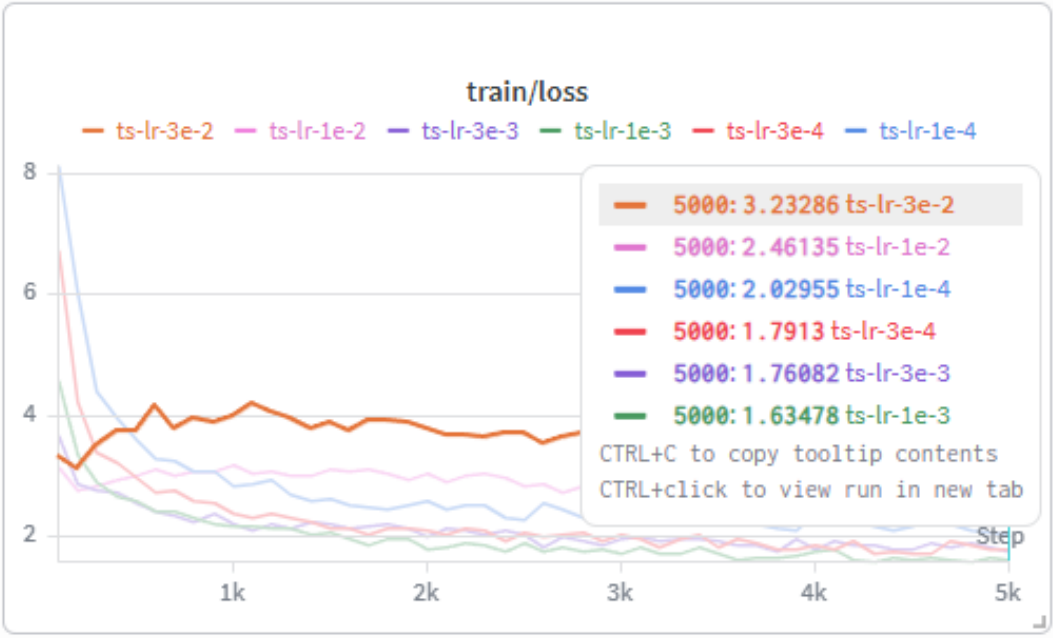
(b) Folk wisdom is that the best learning rate is "at the edge of stability." Investigate how the point at which learning rates diverge is related to your best learning rate.

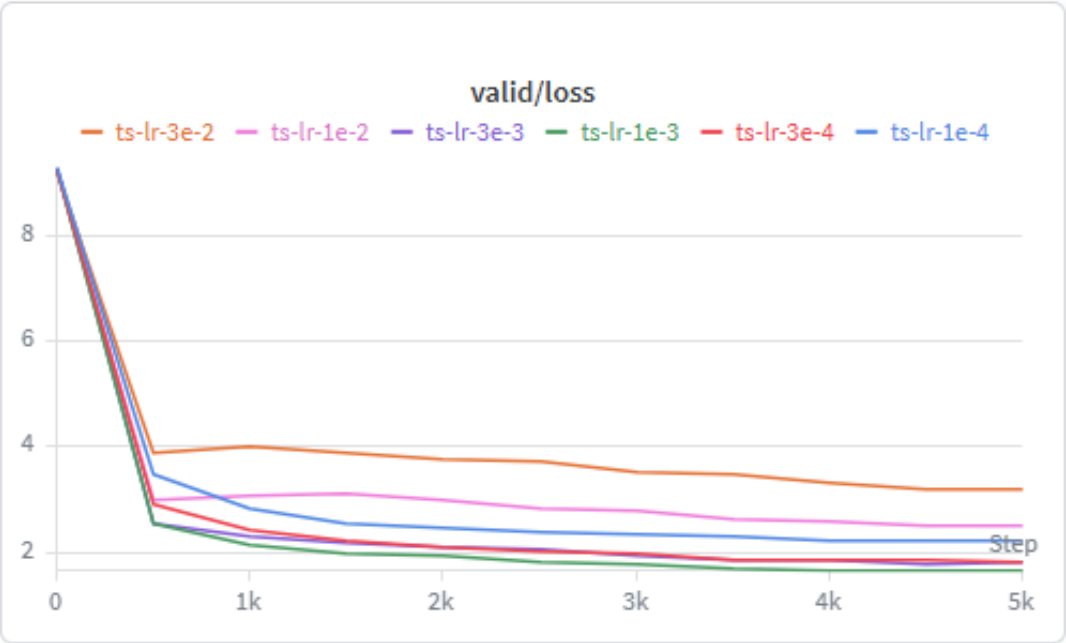
Deliverable: Learning curves of increasing learning rate which include at least one divergent run and an analysis of how this relates to convergence rates.

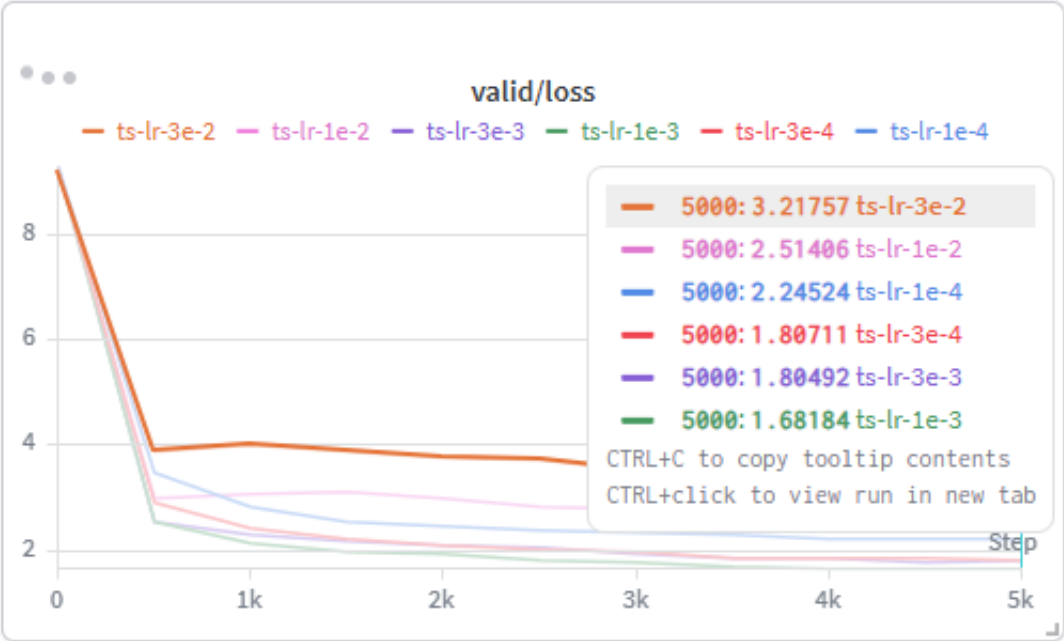
Answer

a. 我首先对学习率做了一个粗粒度的 log-scale sweep，范围从 1e-4 到 3e-2，每个设置训练 5000 steps。这样做的目的是先找到明显过小、合适、以及过大的学习率区间


train/loss
ts-lr-3e-2
ts-lr-1e-2
ts-lr-3e-3
ts-lr-1e-3
ts-lr-3e-4
ts-lr-1e-4
8
6
4
2
Step
1k
2k
3k
4k
5k

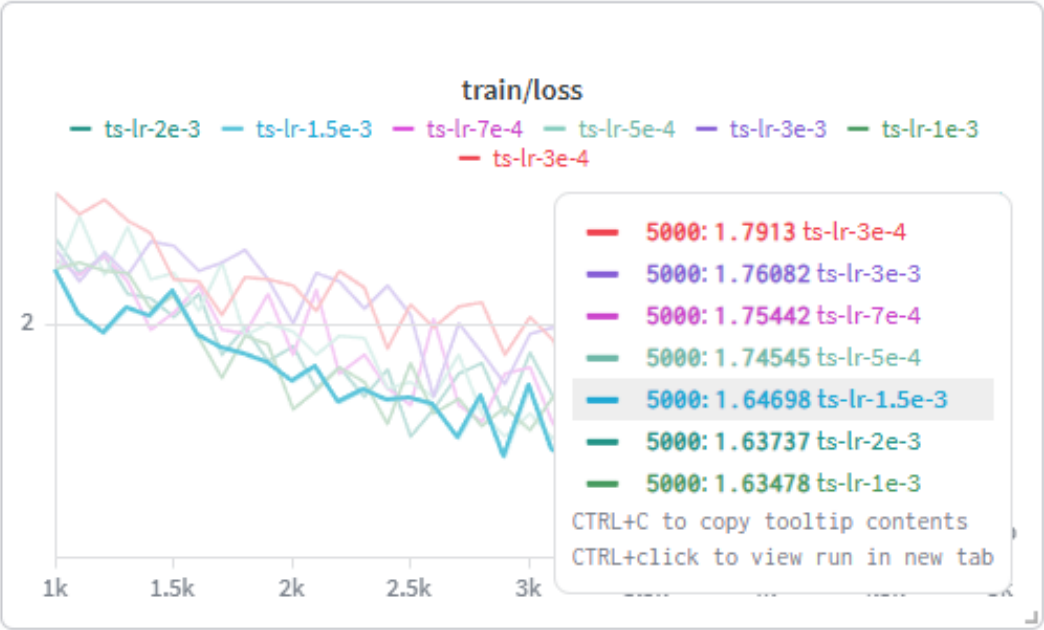

train/loss
ts-lr-3e-2
ts-lr-1e-2
ts-lr-3e-3
ts-lr-1e-3
ts-lr-3e-4
ts-lr-1e-4
8
6
4
2
5000: 3.23286 ts-lr-3e-2
5000: 2.46135 ts-lr-1e-2
5000: 2.02955 ts-lr-1e-4
5000: 1.7913 ts-lr-3e-4
5000: 1.76082 ts-lr-3e-3
5000: 1.63478 ts-lr-1e-3
CTRL+C to copy tooltip contents
CTRL+click to view run in new tab
Step
1k
2k
3k
4k
5k

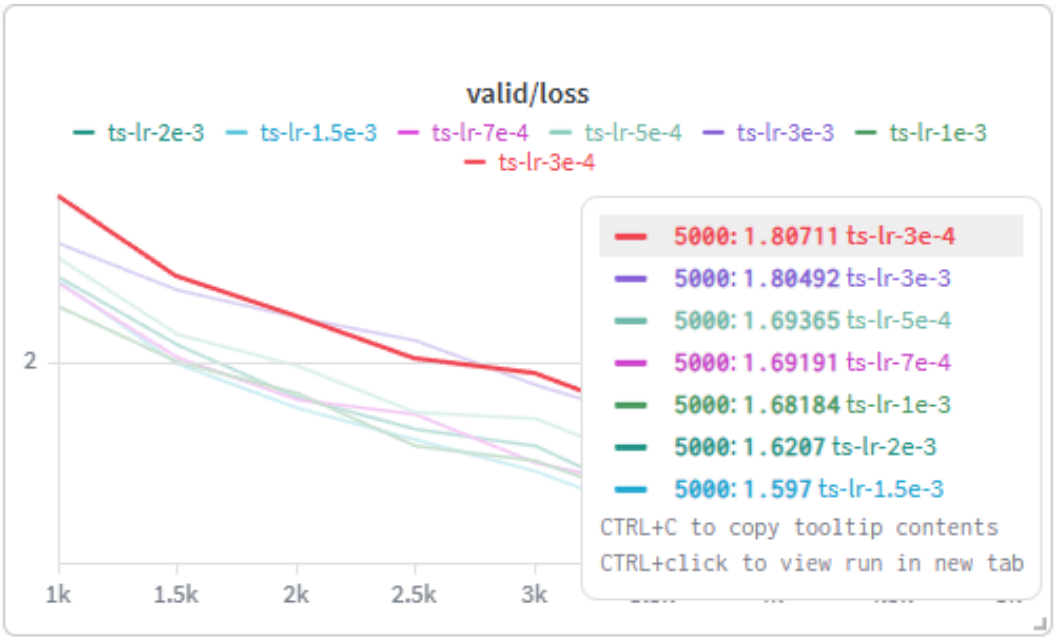




粗扫之后可以看到，1e-4 收敛太慢，1e-2 和 3e-2 的训练效果明显变差，模型不能有效收敛；表现最好的区间大约在 1e-3 附近。

接着我在 3e-4 到 3e-3 之间做了更细的搜索，包括 7e-4、1e-3、1.5e-3 和 2e-3。

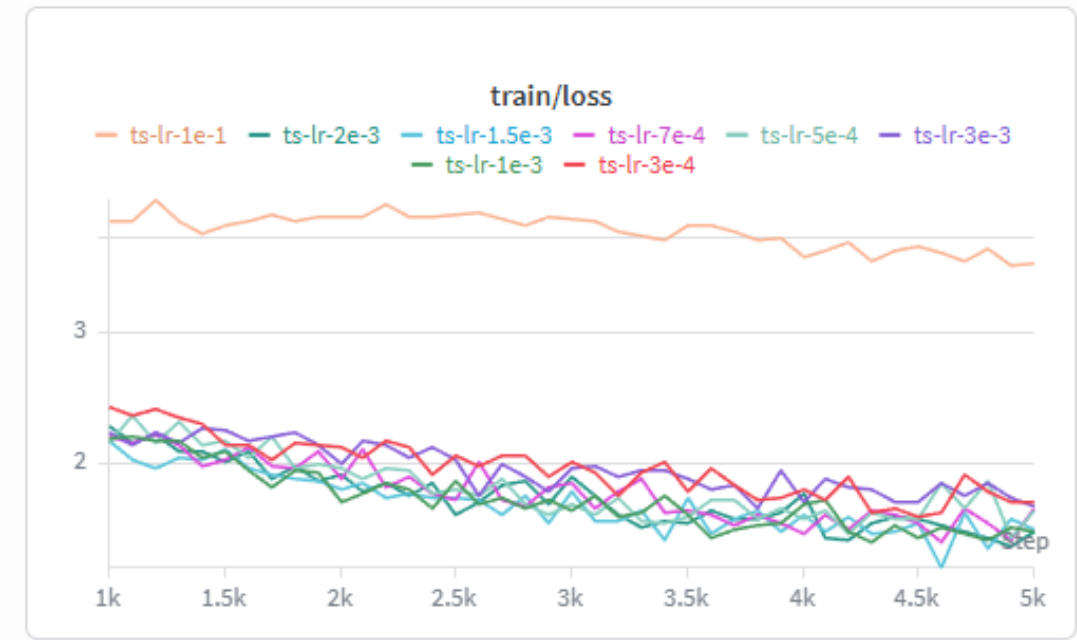


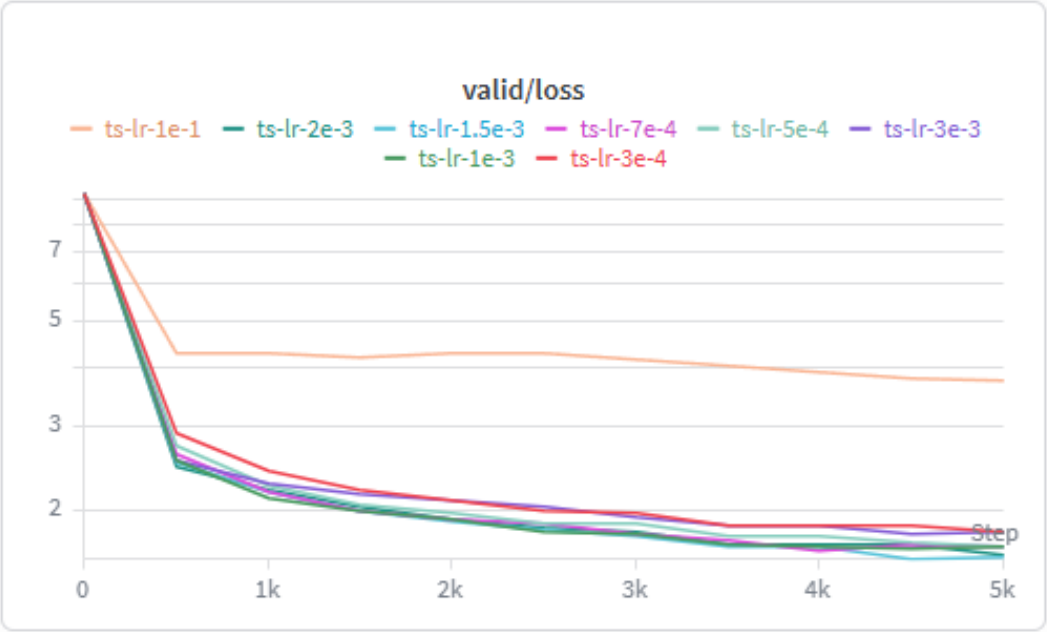


从 validation loss 来看，1.5e-3 在 5000 steps 时表现最好，略优于 1e-3 和 2e-3，因此我选择 1.5e-3 作为最终长时间训练的学习率。

使用 lr = 1.5e-3 训练 40000 steps 后，模型在 TinyStories 上的 validation loss 达到约 1.32 per token，低于 handout 要求的 1.45，说明这个学习率设置能够达到目标性能。

b.





关于 “edge of stability”，实验结果基本符合这个经验规律：当学习率从 1e-4 增大到 1e-3 / 1.5e-3 时，收敛速度明显变快，最终 validation loss 也更低。但继续增大学习率后，优化开始变差：3e-3 仍然可以训练，但效果已经弱于最佳学习率；而 1e-2、3e-2 或更大的学习率不能有效收敛，loss 保持较高或出现明显震荡。因此，最佳学习率并不是越大越好，而是出现在不稳定边界之前、但已经比较接近该边界的位置。

10. Problem (`batch_size_experiment`): Batch size variations (1 B200 hr) (1 point)

Prompt

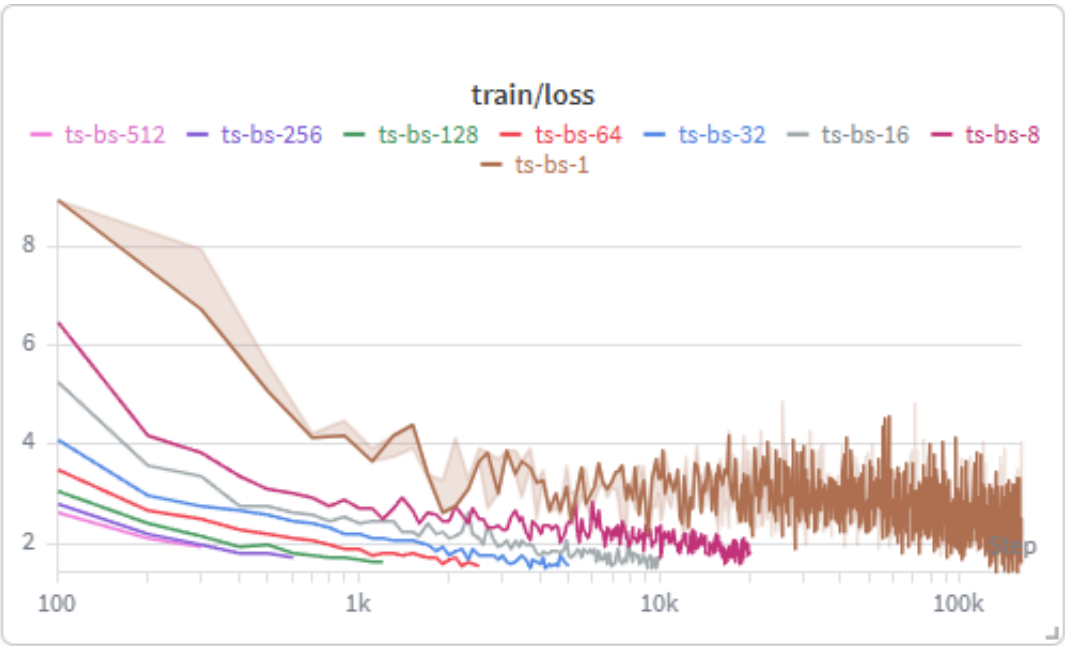
Vary your batch size all the way from `1` to the GPU memory limit. Try at least a few batch sizes in between, including typical sizes like `64` and `128`.

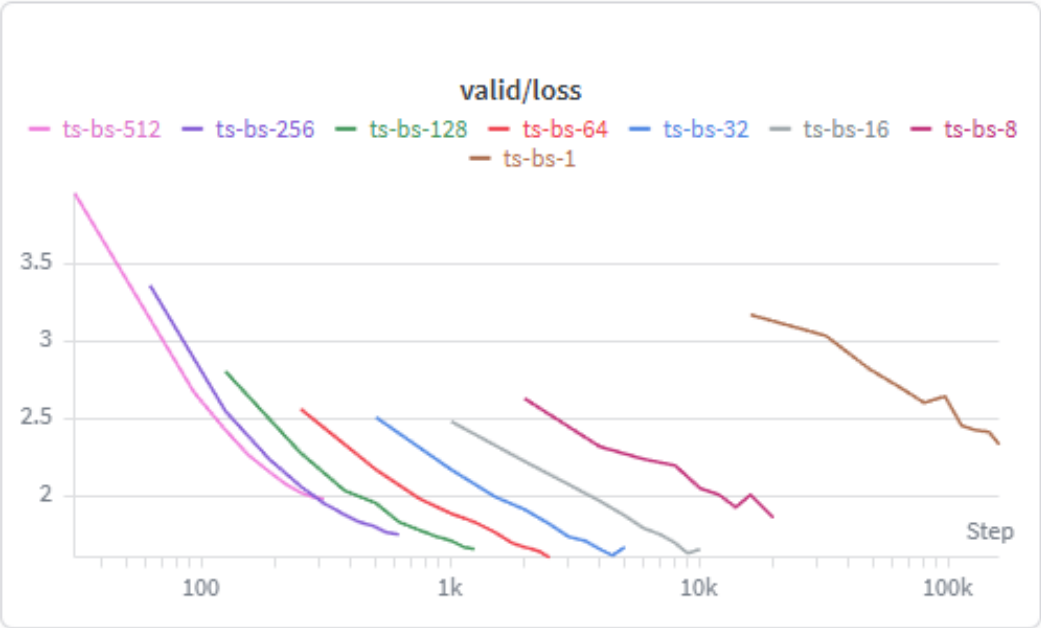
Deliverable: Learning curves for runs with different batch sizes. The learning rates should be optimized again if necessary.

Deliverable: A few sentences discussing your findings on batch sizes and their impacts on training.

Answer

我将 batch size 从 1 一直增大到接近显存上限，并测试了中间的一些典型值，包括 8、16、32、64、128、256 和 512。在我的设置下，最大可用 batch size 约为 624，因此 512 已经接近实际可用的最大 batch size。





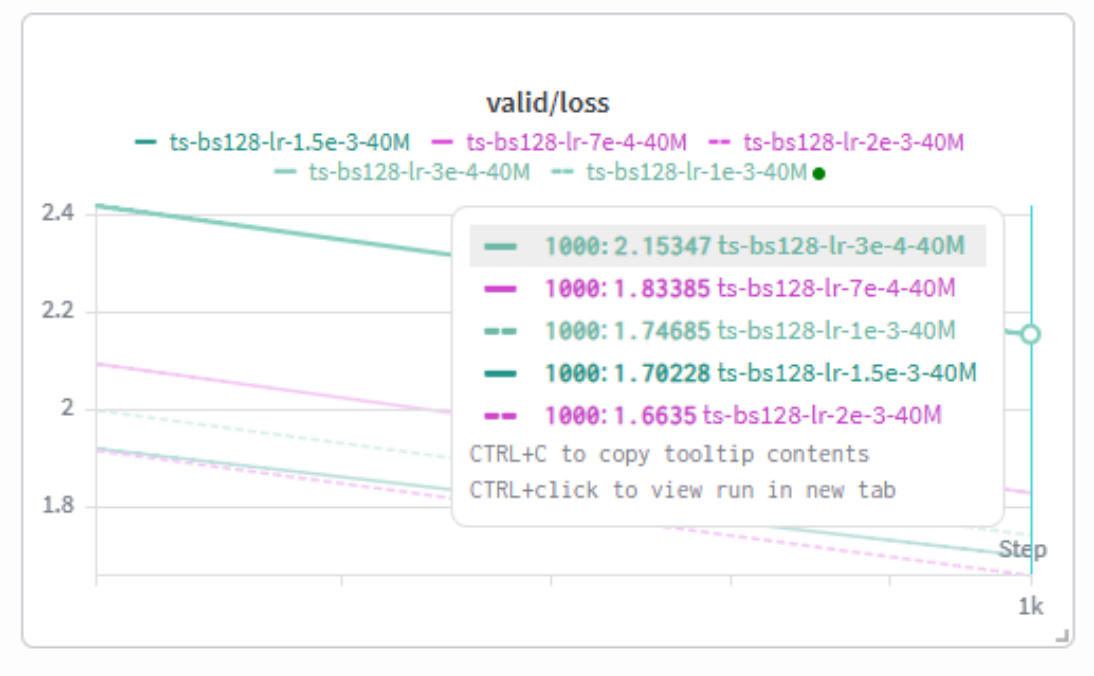
当 batch_size = 1 时，训练曲线非常 noisy，train loss 抖动很明显，而且吞吐只有几千 tokens/s。虽然 validation loss 最后也在下降，但在固定 token budget 下，batch_size=1 需要更多 optimizer updates，而每次 update 的 GPU 利用率又很低，因此 wall-clock 效率很差。batch_size = 8 相比 1 稳定很多，但吞吐仍然明显低于更大的 batch size。

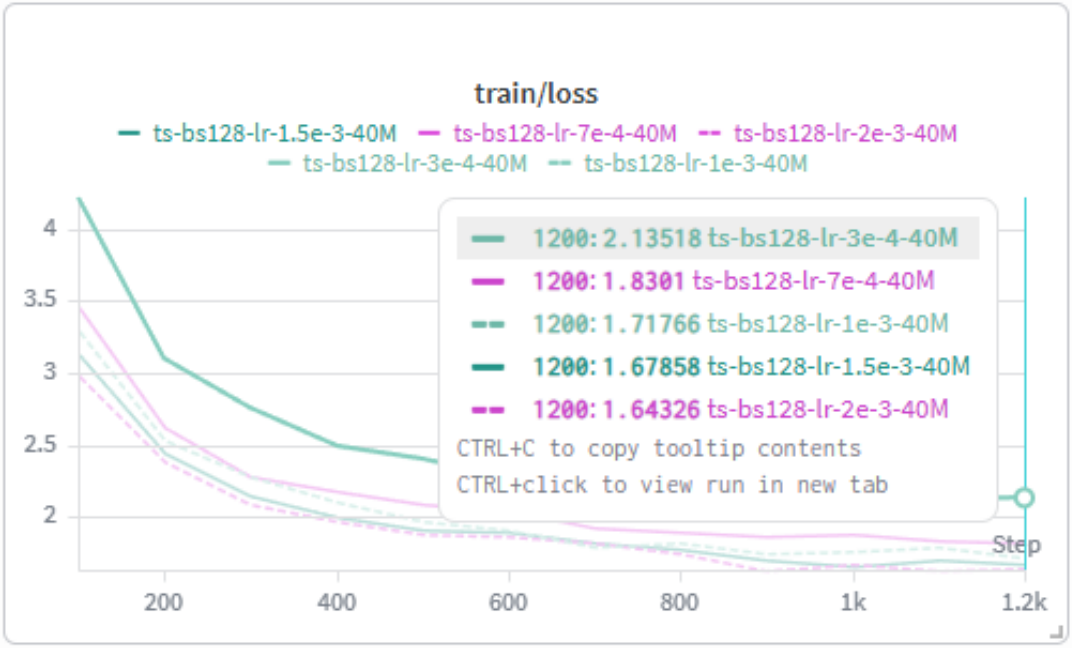
batch_size = 16 到 128 是比较好的区间。这个范围内 train loss 明显更平滑，validation loss 下降也比较稳定，同时吞吐提升到大约 6.5e4 到 8e4 tokens/s。尤其是 batch_size = 128，吞吐最高，validation 曲线也比较顺，因此它在训练效率和优化稳定性之间取得了最好的平衡。

继续增大到 256 或 512 后，吞吐没有继续明显提高，甚至 512 低于 128/256。同时，如果总 token budget 固定，更大的 batch size 意味着 optimizer update 次数更少。虽然每次更新的梯度估计方差更小，但更新次数减少可能会抵消这个优势，所以最大 batch size 并不一定带来更好的 final validation loss。

因此，我后续选择 batch_size = 128。它不是单纯因为 final loss 最低，而是因为它综合来看有最高或接近最高的吞吐、较平滑的训练曲线，以及较好的 validation loss 下降速度。非常小的 batch size 主要问题是噪声大和效率低；非常大的 batch size 则出现吞吐收益递减，并且在固定 token budget 下减少了参数更新次数。

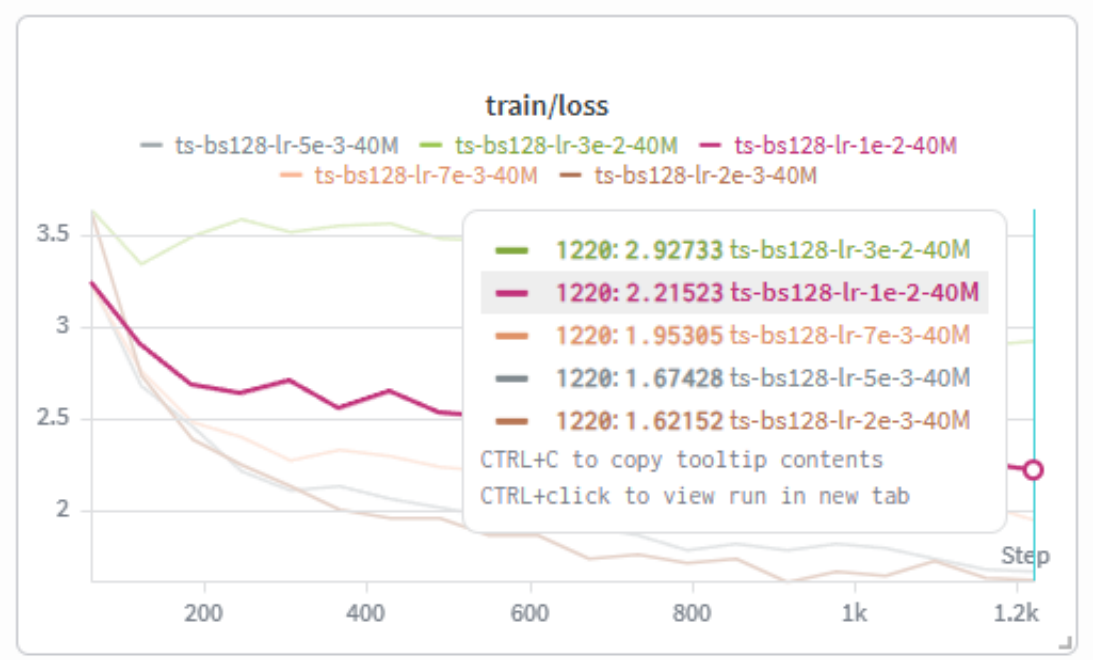
由于 batch size 改成 128 之后，最优 learning rate 可能也会改变，我又在固定 `batch_size = 128`、总训练 token 数为 40M 的设置下重新做了一轮 learning rate sweep。

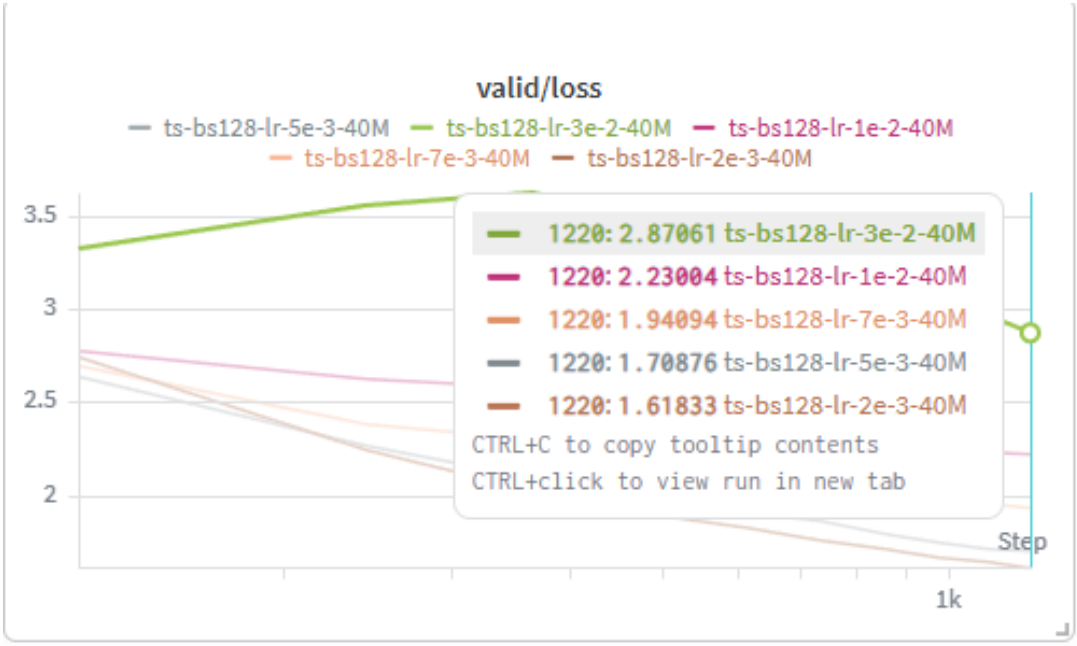




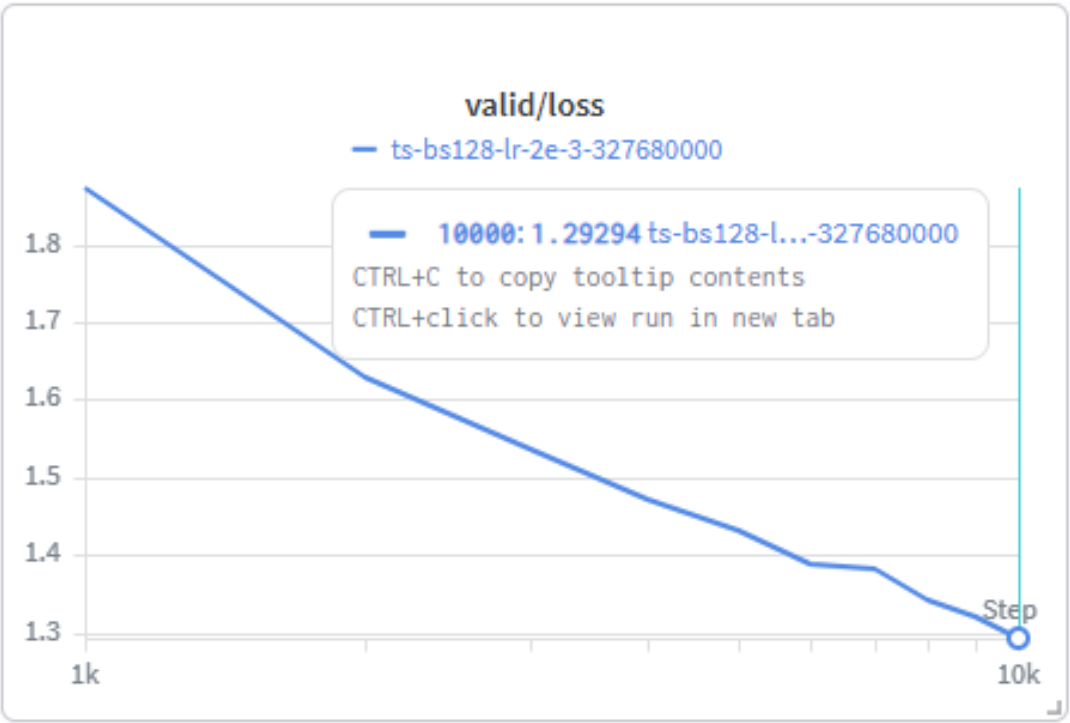
第一轮我测试了 `3e-4`、`7e-4`、`1e-3`、`1.5e-3` 和 `2e-3`。从 train loss 和 validation loss 来看，学习率从 `3e-4` 增大到 `2e-3` 时，收敛速度持续变快，最终 validation loss 也持续降低。其中 `2e-3` 在这组实验中表现最好，40M token 短训结束时的 validation loss 最低。

随后我继续测试比 `2e-3` 更大的 learning rate，以检查 `2e-3` 是否已经接近不稳定边界。结果显示，继续增大学习率后效果开始变差：`5e-3` 虽然仍然可以训练，但 validation loss 已经高于 `2e-3`；`7e-3`、`1e-2` 和 `3e-2` 的训练效果进一步恶化，其中较大的 learning rate 出现明显不稳定或不能有效收敛。





因此，在 `batch_size = 128` 的设置下，我选择 `max_lr = 2e-3` 作为后续实验的学习率。这个结果也说明，在重新选择 batch size 之后重新调 learning rate 是必要的：相比之前的 `1.5e-3`，`2e-3` 在新的 batch size 下取得了更好的短训 validation loss。



在重新选择 `batch_size=128` 并将 `max_lr` 调到 `2e-3` 后，我进行了更长训练。最终 train loss 约为 1.27，valid loss 约为 1.29，优于之前 `batch_size=32, max_lr=1.5e-3, step=40000` 的 valid loss 约 1.32。这说明 batch size 和 learning rate 需要联合考虑；更合适的 batch size 不仅提高吞吐，也可能通过更合适的 learning rate 设置改善最终 validation loss。

11. Problem (`generate`): Generate text (1 point)

Prompt

Using your decoder and your trained checkpoint, report the text generated by your model. You may need to manipulate decoder parameters, `temperature`, `top-p`, and so on, to get fluent outputs.

Deliverable: Text dump of at least `256` tokens of text, or until the first `<|endoftext|>` token, and a brief comment on the fluency of this output and at least two factors which affect how good or bad this output is.

Answer

Once upon a time, there was a little girl named Lily. She had a big, red ball. Lily loved to play with her ball every day. One day, she saw a small, helpless bird on the ground. The bird could not fly. Lily wanted to help the bird. She thought of a plan. She would use her big, red ball to help the bird. Lily threw the ball up in the air, but the bird did not fly. The bird was sad. Then, something unexpected happened. The bird started to grow bigger and bigger! It was not a bird at all, but a big, shiny balloon! The balloon was magic. It could fly and make wishes come true. Lily was so happy. She played with her new balloon all day long. <|endoftext|> Once upon a time, there was a little boy named Tim. Tim had a toy car that he loved to play with. One day, Tim's mom told him to clean his room. Tim did not want to clean his room, but he knew he had to listen to his mom. Tim started to clean his room. He put away his toys and made his bed. But then, something unexpected happened. Tim's toy car started to move on

生成结果整体比较流畅，风格很接近 TinyStories：词汇简单，句子较短，并且有比较清楚的故事结构。模型生成了一个基本完整的情节，包括 Lily 发现小鸟、尝试帮助它、最后出现魔法气球的结尾。不过输出也有一些问题，例如 "big, red ball" 和 "the bird could not fly" 这类表达有重复，且"小鸟变成魔法气球"的转折比较突然，逻辑上略显生硬。

影响生成质量的因素至少有两个。第一是采样参数，例如 temperature 和 top-p；较低的 temperature 会让输出更稳定、更少语法错误，但也可能让故事更保守、更重复。第二是 checkpoint 本身的训练质量，例如训练 token 数、validation loss 和数据/tokenization 质量；训练更充分、validation loss 更低、tokenization 更干净的模型通常会生成更连贯、异常更少的文本。

在生成时我还发现，早期序列化 token ids 的流程没有把 `<|endoftext|>` 编码成 special token，而是把它当作普通字符串拆开了。因此对于这个 checkpoint，我在展示结果时按照第一次出现的 `<|endoftext|>` 字符串进行截断；后续我已经修复了 tokenization 流程。

12. Problem (`layer_norm_ablation`): Remove RMSNorm and train (0.5 B200 hrs) (1 point)

Prompt

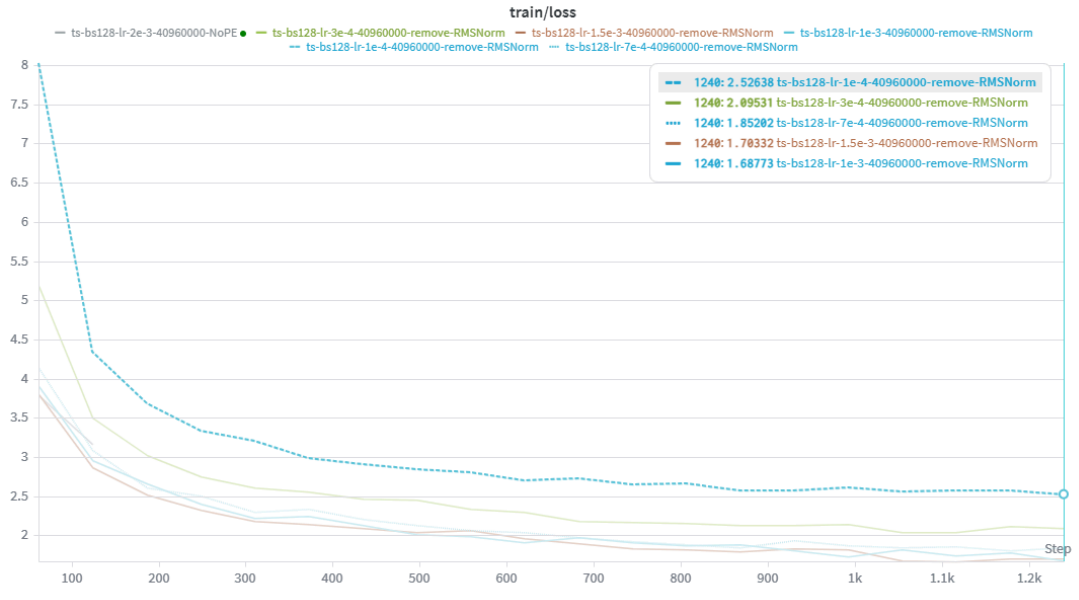
Remove all of the RMSNorms from your Transformer and train. What happens at the previous optimal learning rate? Can you get stability by using a lower learning rate?

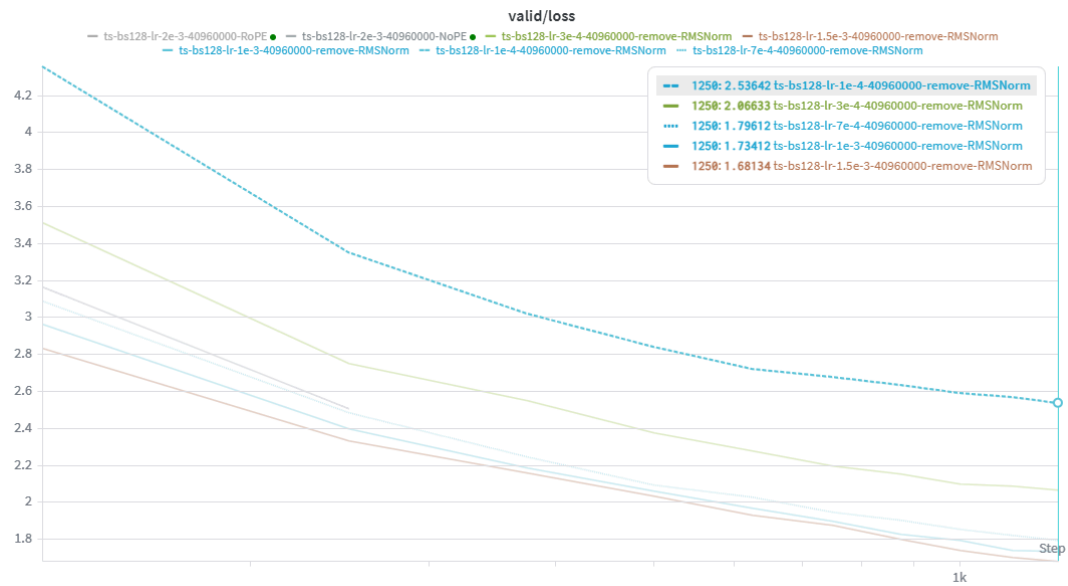
Deliverable: A learning curve for when you remove RMSNorms and train, as well as a learning curve for the best learning rate.

Deliverable: A few sentences of commentary on the impact of RMSNorm.

Answer

移除所有 RMSNorm 后，模型在之前调好的 learning rate `2e-3` 下明显更不稳定，train loss 出现了非常大的 transient spike，说明没有 normalization 时 activation 或 logits 的尺度容易失控。不过该 run 没有直接变成 NaN，后续 loss 仍然可以下降。





当降低 learning rate 后，训练曲线更加稳定，spike 消失。在本次 sweep 中，`1.5e-3` 取得了最低的 validation loss，因此我将其作为移除 RMSNorm 后的最佳 learning rate。

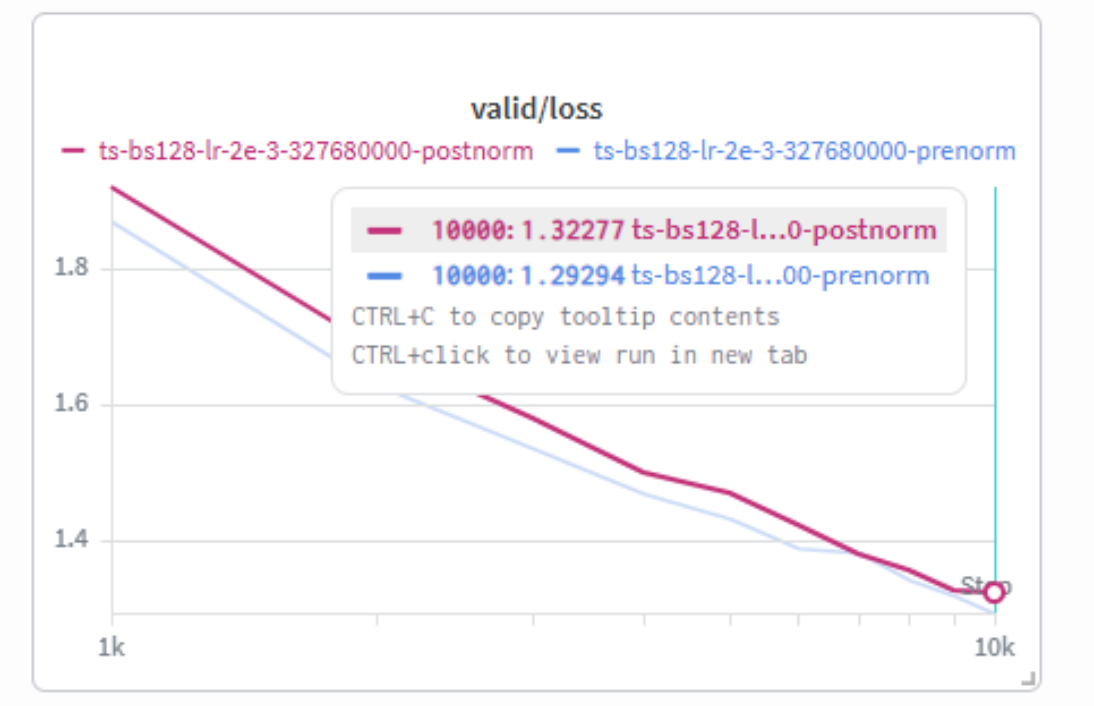
13. Problem (`pre_norm_ablation`): Implement post-norm and train (0.5 B200 hrs) (1 point)

Prompt

Modify your pre-norm Transformer implementation into a post-norm one. Train with the post-norm model and see what happens.

Deliverable: A learning curve for a post-norm Transformer, compared to the pre-norm one.

Answer



我将默认的 pre-norm Transformer 改成 post-norm Transformer，并在相同训练设置下比较。两者使用相同的 batch size、learning rate schedule 和 token budget。

从训练曲线看，两者都能稳定收敛，没有出现明显发散。不过 pre-norm 的 loss 持续略低于 post-norm。到 10k steps 时，pre-norm 的 train loss 约为 1.273，valid loss 约为 1.293；post-norm 的 train loss 约为 1.322，valid loss 约为 1.323。

因此，在本实验的小模型和 TinyStories 设置下，post-norm 仍然可以训练，但表现略差于 pre-norm。这说明 norm 的位置会影响优化效果。pre-norm 可能让每个 sublayer 接收到尺度更稳定的输入，并改善梯度传播；不过由于模型较小、训练任务较简单，这里的差距没有表现为严重不稳定或发散。

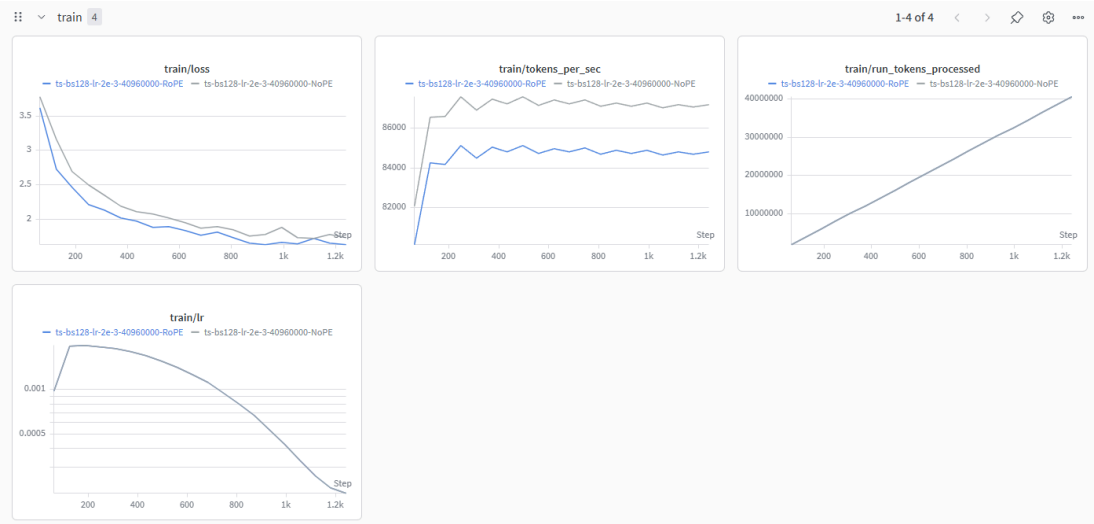
14. Problem (`no_pos_emb`): Implement NoPE (0.5 B200 hrs) (1 point)

Prompt

Modify your Transformer implementation with RoPE to remove the position embedding information entirely, and see what happens.

Deliverable: A learning curve comparing the performance of RoPE and NoPE.

Answer



我比较了使用 RoPE 的 baseline 和完全移除位置编码的 NoPE。两者使用相同的训练设置和 learning rate schedule，只是在 NoPE 中不再对 Q/K 应用 RoPE，causal mask 仍然保留。

从 learning curves 来看，NoPE 仍然可以正常训练，validation loss 持续下降，说明 decoder-only Transformer 确实可以在没有显式位置编码的情况下从 causal mask 和数据分布中学习到一部分位置信息。不过，NoPE 的 train loss 和 validation loss 都明显高于 RoPE，收敛也更慢。RoPE 最终取得了更低的 validation loss，说明显式的相对位置信息对语言建模性能有明显帮助。

NoPE 的 tokens/s 略高，因为省去了 RoPE 计算，但这个计算收益没有弥补 loss 上的性能下降。因此，在本实验设置下，RoPE 比 NoPE 更有效。

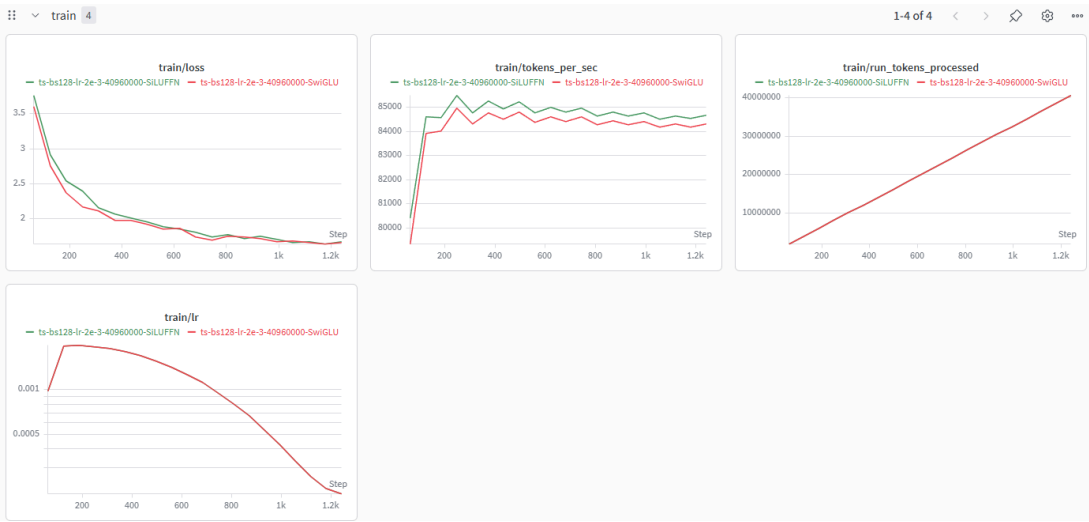
15. Problem (`swiglu_ablation`): SwiGLU vs. SiLU (0.5 B200 hrs) (1 point)

Prompt

Deliverable: A learning curve comparing the performance of SwiGLU and SiLU feed-forward networks, with approximately matched parameter counts.

Deliverable: A few sentences discussing your findings.

Answer



在本次实验中，我比较了默认的 SwiGLU FFN 和参数量大致匹配的 SiLU FFN。SiLU FFN 使用两层线性层，并将 hidden dimension 设置为 `4 * d_model`；SwiGLU 使用三层线性层，hidden dimension 约为 `8/3 * d_model`，因此两者参数量大致相近。

从 learning curves 来看，两种 FFN 的训练曲线非常接近，都能稳定收敛。最终 validation loss 中，SwiGLU 约为 1.616，SiLU FFN 约为 1.629，SwiGLU 略低一些。这说明 gated FFN 在本实验设置下带来了一点性能收益，但差异不大。

一个可能原因是本实验使用的是较小模型和 TinyStories 的短训设置，任务相对简单，因此 FFN 结构差异没有被明显放大。更长训练、更大模型或更复杂数据集上，SwiGLU 的优势可能会更加明显。

16. Problem (`main_experiment`): Experiment on OWT (2 B200 hrs) (2 points)

Prompt

Train your language model on OpenWebText with the same model architecture and total training iterations as TinyStories. You may need to re-tune hyperparameters such as learning rate or batch size.

Deliverable: A learning curve of your language model on OpenWebText. Describe the difference in losses from TinyStories, and how those losses should be interpreted.

Deliverable: Generated text from the OpenWebText language model, in the same format as the TinyStories outputs. Comment on the fluency of this text and why the output quality is worse even with the same model and compute budget as TinyStories.

Answer

TODO: Write your response here.

17. Problem (`leaderboard`): Leaderboard (10 B200 hrs) (6 points)

Prompt

Train a model under the leaderboard rules from the handout, with the goal of minimizing validation loss within `0.75` B200-hours. The run must use only the provided OpenWebText training dataset and fit within a `45` minute wall-clock runtime on a B200.

Deliverable: The final validation loss, an associated learning curve with a wall-clock-time x-axis under `45` minutes, and a description of what you did.

Answer

TODO: Write your response here.